

BranchCraft

互动剧情视频制作器

实验报告

第 25 组

组长：任逍遥 组员：王云琦、唐睿

2026 年 7 月 6 日

摘要

BranchCraft 是一款基于 Qt6/C++ 的互动剧情视频制作工具，由可视化编辑器和独立播放器两大程序组成。编辑器以树形结构组织剧情节点，支持普通节点（含多张幻灯片）和分支节点（含用户选项）；提供场景画布实时预览、属性面板精细编辑、素材库管理、一键导出等功能。播放器加载导出工程，以全屏渲染和鼠标交互呈现最终作品，支持三层音频体系和历史回退。本文详细介绍程序功能、各模块与类的设计细节、小组成员分工及项目总结反思。

目录

1	程序功能介绍	2
1.1	编辑器功能	2
1.2	播放器功能	2
2	项目各模块与类设计细节	2
2.1	整体架构	2
2.2	核心数据模型 StoryGraph	3
2.2.1	编辑器版 (StoryGraph.h/.cpp)	3
2.2.2	播放器版 (PlayerStoryGraph.h/.cpp)	3
2.3	编辑器核心模块	3
2.3.1	MainWindow (主窗口)	3
2.3.2	WelcomeWidget (欢迎界面)	3
2.3.3	StoryGraphScene (剧情树场景)	3
2.3.4	StoryGraphView & NodeGraphicsItem	4
2.3.5	SceneCanvas (场景画布)	4
2.3.6	PropertyPanel (属性面板)	4
2.3.7	辅助模块	4
2.4	播放器核心模块	4
2.4.1	PlayerMainWindow (播放器主窗口)	4
2.4.2	PlayCanvas (播放器画布)	4
2.5	数据流与文件结构	5
3	小组成员分工情况	6
4	项目总结与反思	6
4.1	主要成果	6
4.2	存在的不足	7
4.3	改进方向	7
4.4	心得体会	7

1 程序功能介绍

BranchCraft 是一款基于 Qt6/C++ 开发的互动剧情视频制作器，由编辑器（Editor）和播放器（Player）两大独立程序组成。编辑器以可视化方式编排分支剧情树及各场景幻灯片内容；播放器加载导出工程，以交互方式呈现最终作品。

1.1 编辑器功能

（1）**工程管理**。支持新建、打开、保存、另存、清空工程及一键导出。新建时自动创建工程文件夹（含 `story.json` 和 `resources/`）。导出时将播放器可执行文件、剧情数据和素材复制到目标目录，生成启动批处理文件。

（2）**欢迎界面**。启动后显示欢迎界面，提供“新建项目”“打开已有项目”“功能演示”入口，列出近期工程以便快速恢复工作。

（3）**素材库管理**。左侧面板列出已导入素材（图片、音频、视频）。支持导入、刷新、重命名、删除、替换及双击预览（图片、音频、视频三种模式）。

（4）**剧情树编辑**。上半区域为可视化剧情树，分为**普通节点**（Normal，含多张幻灯片依序播放后跳转子节点）和**分支节点**（Choice，含一张幻灯片和若干分支选项供观众选择跳转）。支持添加子节点、删除、重命名、修改节点类型。节点可拖拽移动/重新建立父子关系，滚轮缩放，Alt+拖拽平移。父子连线自动绘制，分支节点跳转关系以文字标注呈现。

（5）**场景实时预览**。下半区域以 1920×1080 虚拟舞台实时渲染当前幻灯片：背景图片、角色立绘（含名字文本框）、剧情文字框、分支选项按钮。所有元素可拖拽定位，支持图层管理，点击即提升至最上层。

（6）**属性编辑面板**。右侧以选项卡组织：文字（对白、文本框位置/大小/字体/颜色/背景）、背景（背景图片选择）、角色（添加/删除角色，设置名字、字体、颜色、位置、缩放）、媒体（幻灯片音频、停留时长、背景音乐控制）、分支选项（仅分支节点：管理选项列表，编辑按钮文字、字体、颜色、尺寸和跳转目标）。面板与画布双向同步。

（7）**幻灯片管理**。普通节点支持多张幻灯片的添加、删除、复制和切换，每张独立配置。

（8）**预览放映**。按 F5 启动独立播放器预览当前工程。

1.2 播放器功能

（1）**工程加载**。读取命令行参数指定路径下的 `story.json`，加载全部剧情数据。

（2）**场景渲染**。以 1920×1080 虚拟坐标系自适应渲染：背景图、角色立绘与名字、剧情文字、分支选项按钮。

（3）**交互导航**。普通节点左键点击推进幻灯片，播完跳转子节点；分支节点点击选项按钮跳转目标节点；右键返回上一状态（历史栈最多 50 条记录）。

（4）**音频系统**。三层独立音频：节点音频、幻灯片独立音频、背景音乐（支持继续/停止/切换三种行为，可循环播放）。

2 项目各模块与类设计细节

2.1 整体架构

项目分为编辑器（BranchCraft）和播放器（BranchCraftPlay）两个独立 Qt 项目，共享 JSON 数据格式。编辑器以 `MainWindow` 为中枢，通过信号-槽协调各子模块；播放器以 `PlayerMainWindow`

为核心。整体采用 MVC 变体：StoryGraph 为 Model，场景画布/属性面板为 View，操作函数为 Controller。

2.2 核心数据模型 StoryGraph

2.2.1 编辑器版 (StoryGraph.h/.cpp)

数据结构定义：

- **NodeType:** Normal (普通节点)、Choice (分支节点); **BgmAction:** Continue、Stop、Switch。
- **Slide::Character:** 角色名字、立绘路径、位置、缩放，以及名字文本框 (位置、字体、颜色、背景图)。
- **Slide:** 背景图、角色列表、文字、文本框属性 (位置/大小/字体/颜色/背景)、时长、图层顺序、背景音乐行为和路径、幻灯片音频。
- **NodeData::Choice:** 按钮文字、背景图片、目标节点 ID、按钮位置、字体大小/颜色、按钮尺寸。
- **NodeData:** 节点 ID、类型、名称、幻灯片列表、音频路径、选项列表、父/子节点 ID。

主要操作：**addNode()** 建立父子关系，**removeNode()** 递归删除子树并清理跨引用，**updateNode()** 保留父子关系更新数据，**moveNode()** 移动到新父节点，**toJson()/fromJson()** 序列化与反序列化。使用 **QRecursiveMutex** 保证线程安全，**rebuildChildIds()** 修复数据一致性。信号 **dataChanged()** 触发界面刷新。

2.2.2 播放器版 (PlayerStoryGraph.h/.cpp)

简化版，仅含 **getNode()**、**getAllNodes()**、**getRootId()**、**fromJson()** 等只读操作，无需锁和写方法。

2.3 编辑器核心模块

2.3.1 MainWindow (主窗口)

继承 **QMainWindow**，编辑器中枢控制器。构建文件/编辑/节点/视图四组菜单和两个工具栏。中央区域用 **QStackedWidget** 切换欢迎界面与编辑器界面。编辑器界面为水平分割器布局：**MaterialWidget | (StoryGraphView | SceneCanvas) | PropertyPanel**。管理约 30 条信号-槽连接，涵盖素材库操作、节点选中联动、数据变更刷新、画布拖拽回传等。提供 **loadStoryFromJson()** 和 **saveStoryToJson()** 实现 JSON 读写 (使用 **QSaveFile** 原子写入)，**onExportProject()** 一键导出 (递归复制播放器 + 数据 + 素材)，**onPreview()** 通过 **QProcess::startDetached()** 启动播放器。

2.3.2 WelcomeWidget (欢迎界面)

居中面板，包含标题、副标题、新建/打开/演示按钮、近期工程列表。通过信号与 **MainWindow** 通信。**updateRecentProjects()** 从 **ProjectManager** 获取路径，解析文件夹名显示。

2.3.3 StoryGraphScene (剧情树场景)

继承 **QGraphicsScene**，管理 **NodeGraphicsItem** 和连线。**buildScene()** 遍历所有节点创建图元并绑定信号。**layoutTree()** 递归树形布局，自底向上计算子树宽度分配坐标。**updateConnectionsAndJumpIndi**

绘制父子连线和跳转标注。`onNodeDragFinished()` 检测拖拽：若靠近其他节点则重新建立父子关系，远离则断开连接。`isDescendant()` 防止环形引用。

2.3.4 StoryGraphView & NodeGraphicsItem

`StoryGraphView`（继承 `QGraphicsView`）提供 `Ctrl+` 滚轮缩放（ $0.1\times$ 至 $5.0\times$ ）、`Space` 适应窗口、`Alt+` 拖拽平移、右键菜单（删除/重命名/修改类型）。`NodeGraphicsItem` 双重继承 `QObject` 和 `QGraphicsRectItem`，绘制节点名称、类型圆点（绿 = 普通/黄 = 分支）、加号按钮（添加子节点）；选中时蓝底蓝框；支持拖拽移动和加号菜单操作。

2.3.5 SceneCanvas（场景画布）

继承 `QWidget`，在 1920×1080 虚拟舞台渲染幻灯片。`updateFromNode()` 加载背景/角色/文字/选项。`paintEvent()` 按 `m_drawOrder` 图层顺序绘制。实现四种拖拽类型（角色图片位置、角色名字、文本框、选项按钮），`bringToFront()` 提升图层，`saveItemPositions()` 写回 `StoryGraph`。`updateDrawRect()` 自适应缩放，黑边填充。

2.3.6 PropertyPanel（属性面板）

最复杂组件（约 1550 行），含节点名称编辑框和幻灯片导航栏。五个选项卡：文字（对白编辑、位置/大小/字体/颜色/背景）、背景（图片选择）、角色（列表管理、名字样式、位置/缩放）、媒体（音频、时长、背景音乐控制）、分支选项（动态显示：选项列表管理、按钮属性编辑、跳转目标设置）。使用 `m_loading` 标志和 `QSignalBlocker` 防止信号重入。提供 `updateTextRect()`、`updateCharacterNameRect()` 等回调实现画布到面板的同步。

2.3.7 辅助模块

MaterialWidget：列表显示素材（存相对路径），提供导入/刷新/右键菜单（重命名/删除/替换），按后缀过滤供属性面板使用。**ProjectManager**：单例模式，管理工程创建（文件夹结构 + 默认 JSON）、打开/保存、近期工程（`QSettings` 持久化最多 10 条）。**NewProjectDialog**：工程名称与路径输入。**PreviewDialog**：根据文件类型自动选择图片显示/音频播放/视频播放。**StyleSheets**：命名空间存储欢迎界面和主窗口的 `QSS` 样式。

2.4 播放器核心模块

2.4.1 PlayerMainWindow（播放器主窗口）

加载 `story.json`，定位根节点开始播放。`setCurrentNode()` 切换节点/幻灯片并处理历史/音频/场景更新。`nextSlide()` 推进幻灯片或跳转子节点。音频系统为三层独立 `QMediaPlayer`（节点音频、背景音乐（支持循环）、幻灯片音频），各配 `QAudioOutput`。`handleBgmAction()` 实现继续/停止/切换逻辑。历史栈用 `QVector<QPair<QString,int>>` 记录访问轨迹（最多 50 条），右键回退。

2.4.2 PlayCanvas（播放器画布）

继承 `QWidget`，渲染逻辑与编辑器 `SceneCanvas` 类似但为只读。`updateFromNode()` 加载数据，`paintEvent()` 按图层绘制，`mousePressEvent()` 左键判断分支选项区域——是则发射 `choiceSelected`，否则发射 `canvasClicked`；右键发射 `historyBackRequested`。

2.5 数据流与文件结构

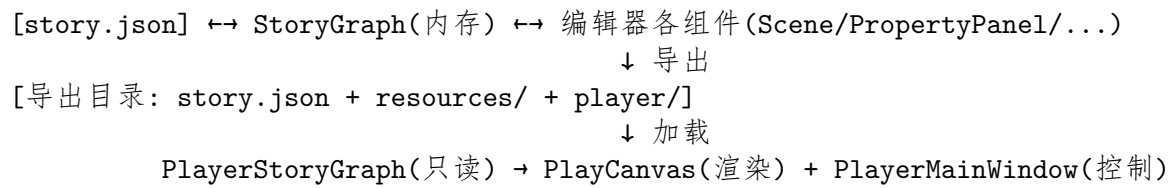


图 1: 数据流简图

工程目录结构: `project.ist`(元数据)、`story.json`(顶层结构 `{"rootId": ..., "nodes": [...]}`), 每个节点含 `id`、`type`、`name`、`slides`、`choices`、`parentId` 等字段)、`resources/`(素材文件目录)。

3 小组成员分工情况

表 1: 小组成员分工表

姓名	角色	负责内容
任逍遥	组长	编辑器主窗口各部分制作: - 主窗口框架 (MainWindow): 菜单栏、工具栏、属性面板、中央区域布局 - 核心数据模型 (StoryGraph): 节点树数据结构设计与 JSON 序列化 - 剧情树可视化 (StoryGraphScene/View、NodeGraphicsItem): 图形化显示、树形布局算法、拖拽交互、重连、父子连线 - 场景画布 (SceneCanvas): 幻灯片实时预览渲染、元素拖拽定位、图层管理 - 属性面板 (PropertyPanel): 五页选项卡编辑、幻灯片导航、双向数据同步 - 素材库 (MaterialWidget): 素材列表展示、导入/删除/替换管理 - 工程管理器 (ProjectManager): 单例模式、工程 CRUD、近期工程记录 - 导出功能 (播放器 + 数据 + 素材一键打包)、实时放映 (QProcess 启动外部播放器) - 各组件间约 30 条信号-槽连接与整体协调
王云琦	组员	播放器部分制作: - 播放器主窗口 (PlayerMainWindow): 工程加载、场景导航、交互控制 - 播放器画布 (PlayCanvas): 场景渲染、鼠标交互 (左键推进/分支选择, 右键回退) - 播放器数据模型 (PlayerStoryGraph): 只读 JSON 解析 - 音频系统: 节点音频/幻灯片音频/背景音乐三层体系 (含 BGM 继续/停止/切换逻辑) - 历史回退栈: 最多 50 条记录的导航回溯 - 命令行参数解析、与编辑器导出格式的对接调试
唐睿	组员	欢迎界面制作和后续功能完善: - 欢迎界面 (WelcomeWidget): 新建/打开/演示按钮、近期工程列表及刷新 - 新建工程对话框 (NewProjectDialog): 工程名称与路径输入 - 素材预览对话框 (PreviewDialog): 图片/音频/视频三种预览模式 - 全局样式表 (StyleSheets): 欢迎界面和主窗口的 QSS 样式设计与美化 - 程序整体测试、Bug 修复与后续功能完善

4 项目总结与反思

4.1 主要成果

- (1) **数据模型设计。**以 StoryGraph 为核心的树形数据结构将普通节点和分支节点统一表示，兼顾了多幻灯片、角色管理、分支选项、背景音乐控制和图层排序等需求。JSON 格式在编辑器和播放器之间保持一致，导出后可直接播放，未出现格式兼容问题。
- (2) **交互操作。**剧情树支持拖拽节点改变父子关系和树形布局，场景画布中角色、文本框和按钮均可直接拖拽定位，属性面板与画布之间保持双向同步（通过信号阻塞和加载标志防止重入）。这些交互设计减少了手动输入数值的频率。

(3) **模块划分**。素材库、画布、属性面板等组件各司其职，通过信号-槽机制通信，组件之间耦合度较低。每个模块的修改不容易影响其他模块，小组成员在不同文件中工作时的代码冲突较少。

(4) **导出功能**。导出时将播放器可执行文件及其依赖库与工程数据和素材一并打包到目标目录，用户执行生成的批处理文件即可播放，无需单独安装 Qt 运行环境。

4.2 存在的不足

(1) **工程另存为功能未完成**。`ProjectManager::saveAsProject()` 仅保留了函数框架，未实现完整的工程复制逻辑，用户需要手动复制文件夹进行备份。

(2) **视频媒介支持有限**。素材库可导入视频文件，预览对话框也能播放，但幻灯片不支持视频作为背景或角色元素。主要原因是在画布渲染流程中嵌入视频播放需要处理 `QMediaPlayer` 与 `QPainter` 的协调，受开发时间限制未能实现。

(3) **缺少撤销与重做功能**。编辑器中节点的增删、拖拽、属性修改等操作均不可撤销，误操作后只能手动恢复，在一定程度上降低了编辑效率。

(4) **播放器功能较简单**。当前播放器主要提供逐张幻灯片点击推进和分支选项跳转功能，缺少自动播放、进度条、存档读档等常见功能。

(5) **未进行跨平台测试**。项目仅在一台 Windows 环境下完成开发与测试，导出包中也包含 bat 脚本等 Windows 专用文件。在 macOS 或 Linux 系统上运行可能需要进行适配。

4.3 改进方向

1. 为编辑器添加撤销/重做功能，至少覆盖节点增删和属性修改操作。
2. 补全工程另存为功能，并考虑加入自动保存机制。
3. 支持视频素材作为幻灯片背景或角色元素。
4. 为播放器添加自动播放模式、进度条以及存档功能。
5. 在 macOS 和 Linux 平台上进行编译测试与适配。

4.4 心得体会

本项目是小组第一次使用 Qt 框架完成一个较为完整的桌面应用程序，从设计到实现过程中积累了一些经验。

在接口约定方面，小组在开发初期确定了 JSON 数据格式中各个字段的含义和结构，这使得编辑器和播放器可以并行开发，后期整合时未出现大的格式冲突，节省了对接时间。

在组件通信方面，Qt 信号-槽机制在各模块解耦上起到了较大作用，但随着功能增多，`Main-Window` 中的连接数量也不断增长（约三十条），导致个别数据变更会触发多个位置的级联刷新，增加了调试的复杂度。今后在类似项目中可能需要更系统地规划信号流动路径。

在实现细节方面，部分功能的开发难度超过了预期。例如属性面板中为防止信号循环触发而加入的 `QSignalBlocker` 和 `m_loading` 标志、画布自适应缩放与图层排序等，虽然在需求中描述简单，但实际编码和调试花费了较多时间。

总体而言，本项目从零开始完成了一个含编辑器和播放器的互动剧情视频制作工具，虽然尚有改进空间，但通过这次实践，小组对 Qt 开发、C++ 面向对象设计和团队协作有了更具体的认识。